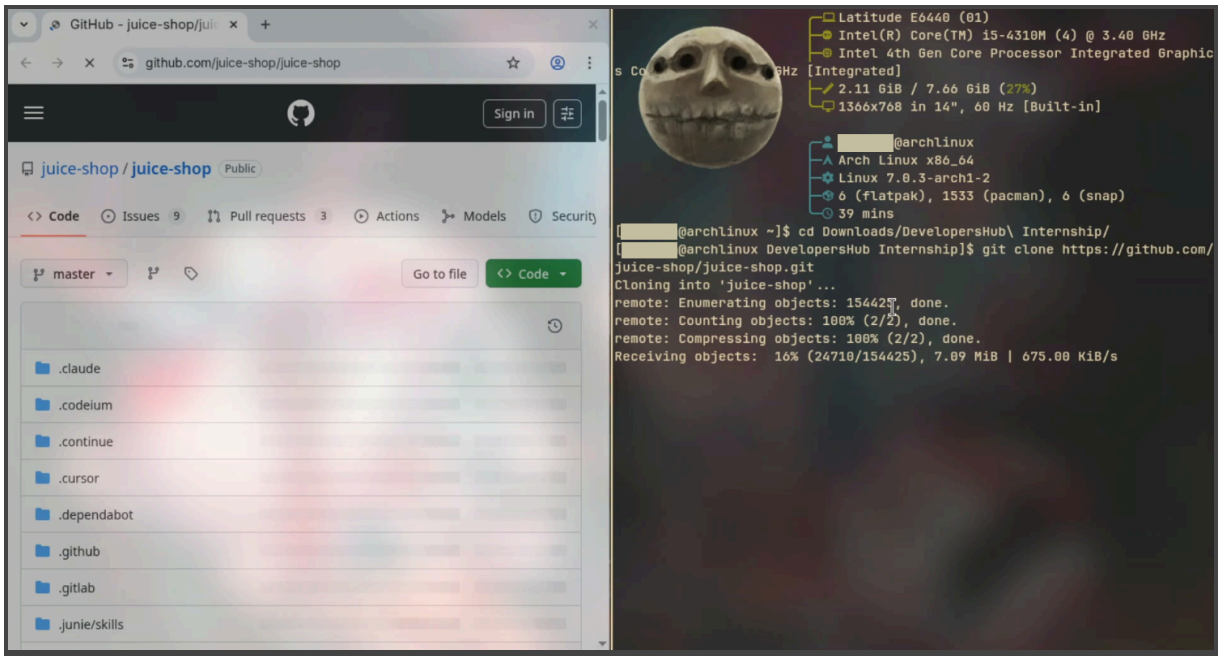
Week 1 – Red Teaming

1.0 Setup Process

1.1 Installing Web Files

This assessment tasks us to install web files through github. I completed the setup process by using the official Juice Shop repository from GitHub by using the command:

git clone https://github.com/juice-shop/juice-shop.git



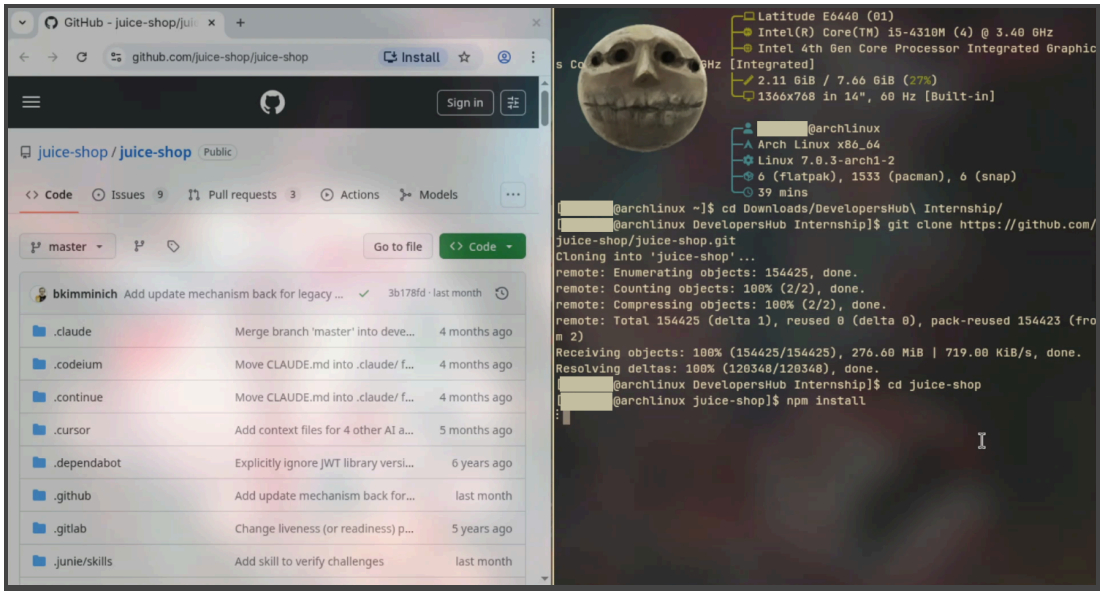
(Figure 1.0)

Initially, the repository was cloned into the local machine using Git, after that I navigated into the directory, then from there I installed all the packages and dependencies using the command:

npm install

When the files are cloned from github, we also have to fully load the cloned files onto the local server. Hence, I used *npm install*.

Figure 1.0 shows how I used *npm install* on my terminal on the right hand side, while on the left, the Juice Shop repository files are open.

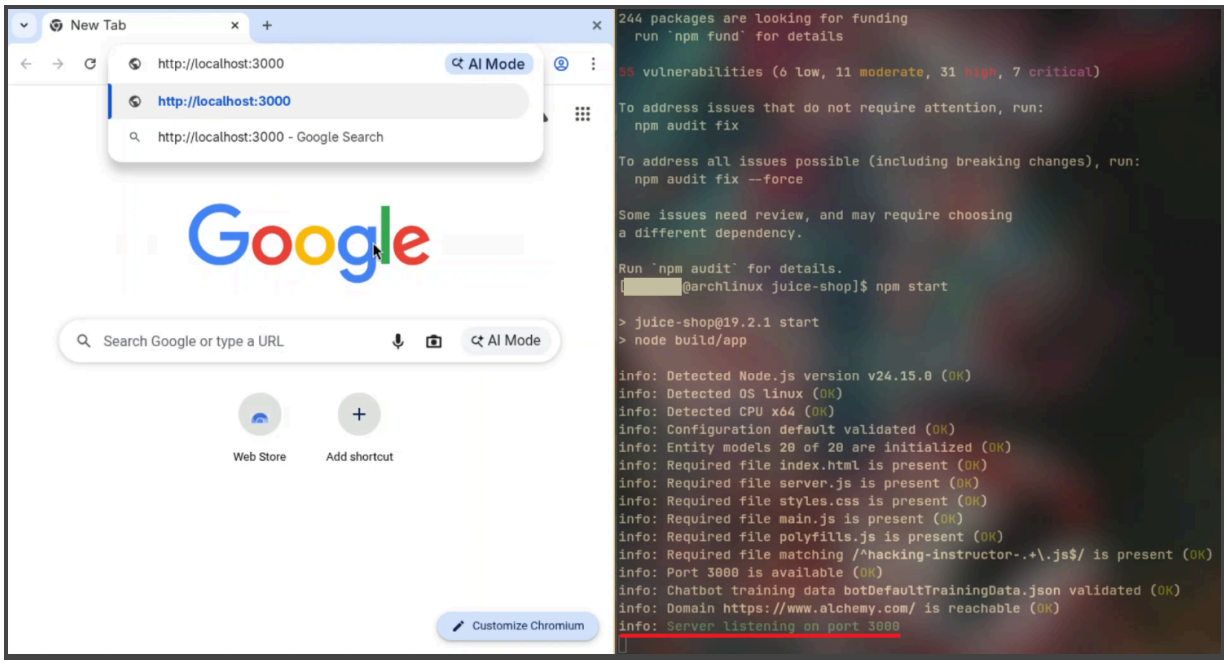


(Figure 1.1)

After it was installed, the application was launched using the command:

npm start

Figure 1.2 shows how I used npm start. The underlined text indicates that the local host has started. On the left side, I am about to enter the URL for the local host.

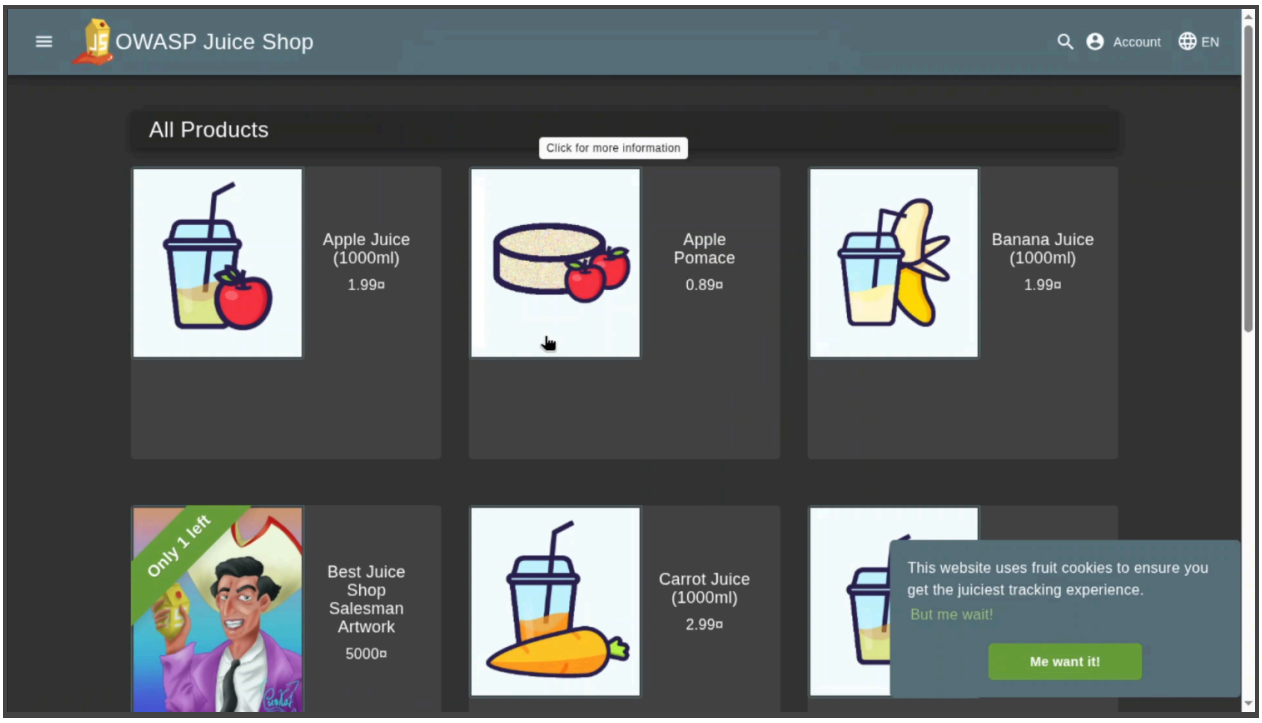


(Figure 1.2)

However, during the installation phase I had encountered some deprecation warnings however, I realised that these warnings were related to older package versions used by some dependencies and are commonly encountered in Node.js projects. So they did not interrupt or affect the installation and execution of the application. Hence, I was able to enter the following URL (where all the installed files from juice-shop are being hosted on)

http://localhost:3000

The setup was then successful, and I was able to view all the pages properly.



(Figure 1.1.3)

2.0 Performing Basic Vulnerability Tests

After I had successfully set up the Juice Shop application and configured the Burp Suite, I then proceeded with performing a basic vulnerability assessment on the web application. The purpose of this assessment was that I had to identify the common web application security vulnerabilities like SQL Injection and Cross Site Scripting (XSS).

2.1 SQL Injection Test

2.1.1 What is SQL Injection?

SQL Injection (SQLi) is a web security vulnerability where malicious SQL statements are inserted into user input fields. If the application fails to properly validate or sanitise the input, attackers may be able to manipulate database queries to bypass authentication, access unauthorised information, or modify database contents

2.1.2 Test Performed

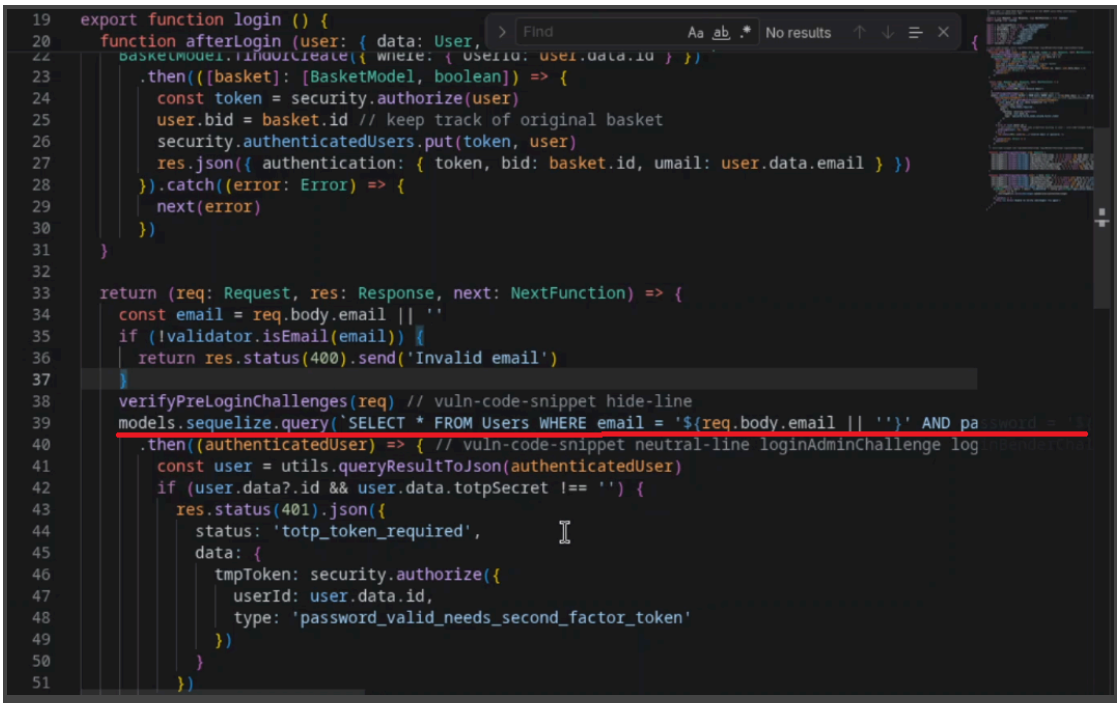
The target page tested was:

http://localhost:3000/#/login

This was the following payload I inserted into the email field:

' OR 1=1–

Figure 2.1.1 shows the line that I identified which took the input from the user (Using Code, the Open Source version for VS code):



(Figure 2.1.1)

The full text for that line contained the following:

models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email || ''}' AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true })

The reason why this line is vulnerable is:

'${req.body.email || ''}'

This is because the user's input becomes part of the SQL command itself. Hence, using *' OR 1=1 --* allows for a direct value to be entered, hence, allowing for the user to skip authentication. To summarise, the application directly inserted user-controlled input (*req.body.email* and *req.body.password*) into an SQL query using string interpolation. This made the login functionality vulnerable to SQL injection attacks.

2.2 Cross Site Scripting (XSS) Test

2.2.1 What is Cross Site Scripting?

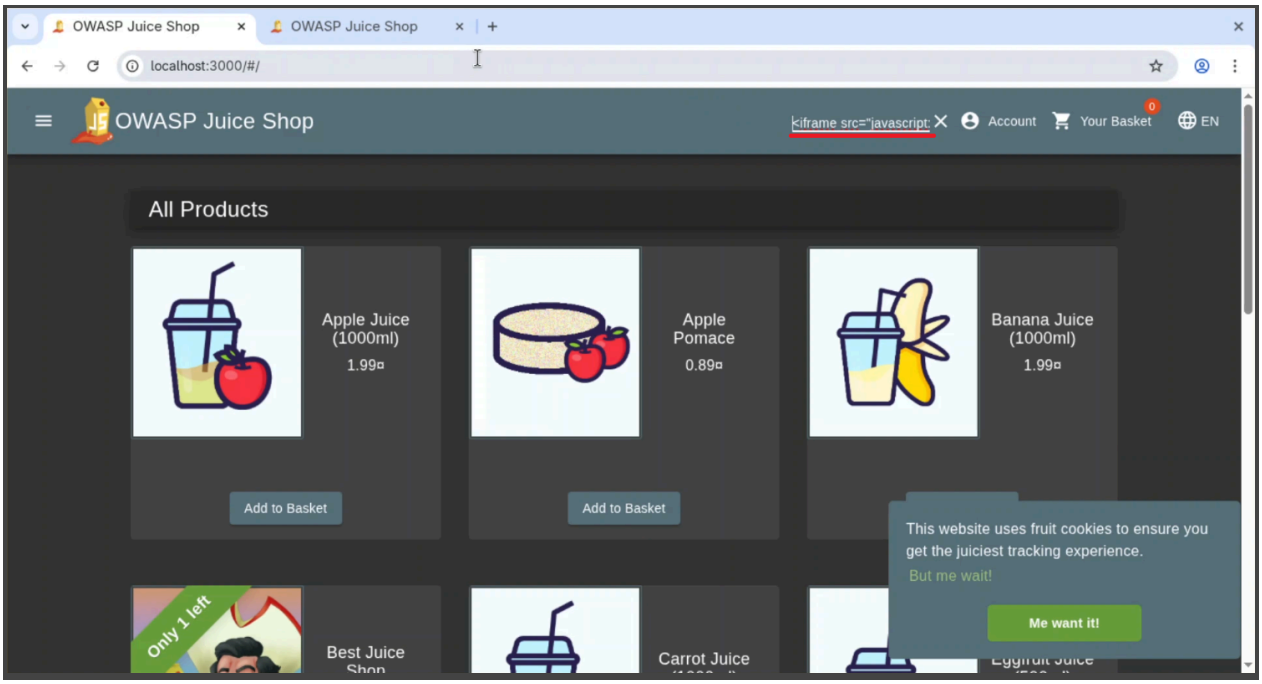
Cross-site scripting (XSS) is a type of web security vulnerability in which an attacker injects malicious scripts, usually JavaScript, into a trusted website. When other users visit the affected page, the malicious script runs in their browser without their knowledge.

XSS attacks are commonly used to steal cookies, session tokens, or personal information, and can also be used to modify website content or redirect users to harmful pages. These attacks happen when a web application does not properly validate or sanitize user input before displaying it.

In order to showcase that the vulnerability, I decided to use the following in the search bar of juice shop:

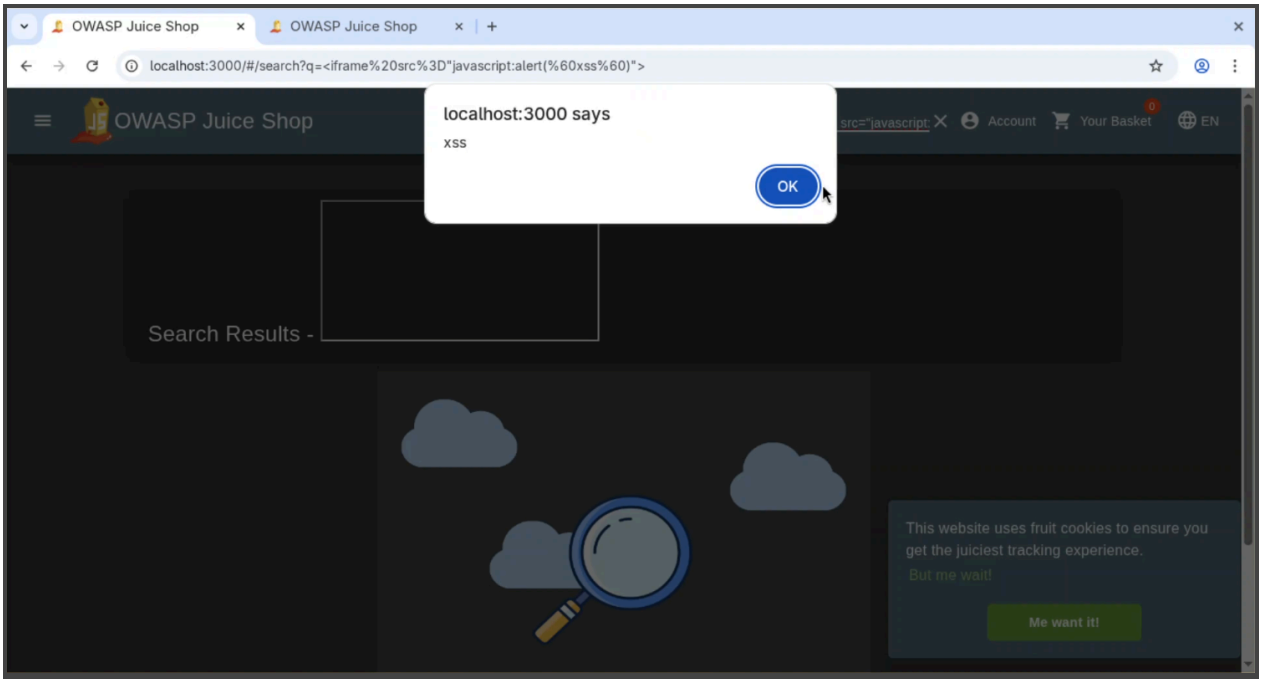
<iframe src="javascript:alert(`xss`)">

Figure 2.2.1 shows how I entered the exact payload as mentioned above in the search bar, underlined by the red line.

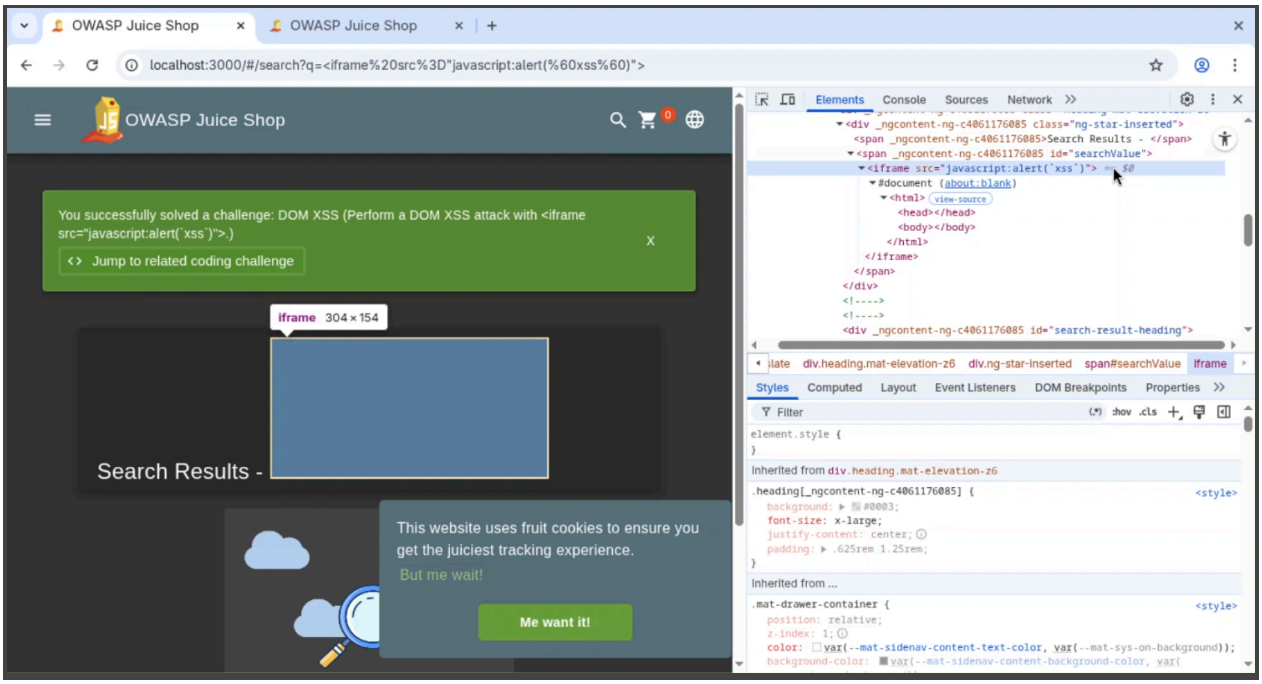


(Figure 2.2.1)

After entering this value, I received a pop-up message (or a java alert) that printed:



(Figure 2.2.2)



(Figure 2.2.3, Using F12 To Show that the payload is the Box in front of Search Results)

Hence, this shows that if a payload that actually contained malicious code was inserted instead of an alert message that prompted *"xss"*, the website could easily become prey to an XSS Attack.

2.3 Using OWASP ZAP to Find Vulnerabilities

2.3.1 Explaining The Purpose of OWASP ZAP

OWASP ZAP is a tool which finds vulnerabilities within a website and how there could be a potential threat from adversaries. It is mainly used by developers, testers, and security professionals to improve the security of websites and web apps before they are released.

2.3.2 Utilising OWASP ZAP

I installed OWASP ZAP application using:

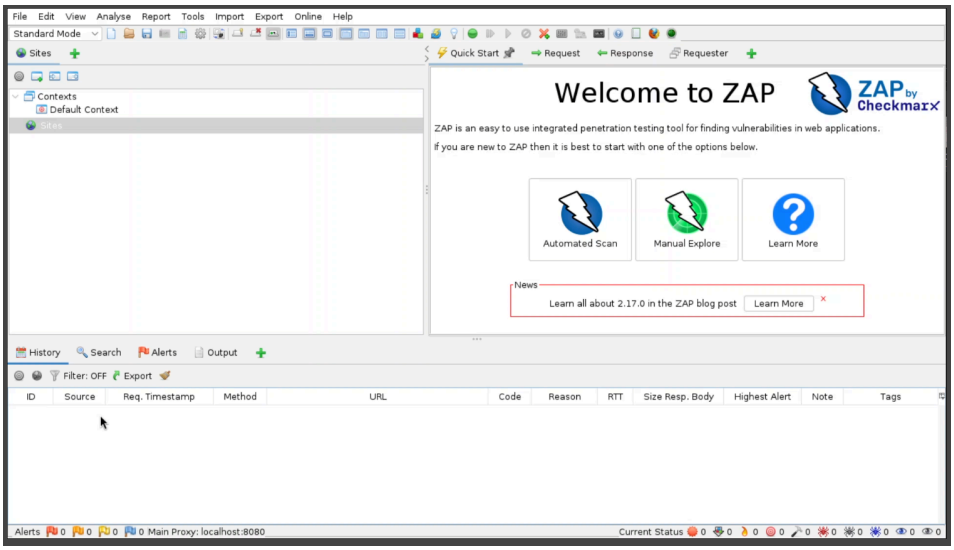
flatpak install flathub org.zaproxy.ZAP

And ran OWASP ZAP using:

flatpak run flathub org.zaproxy.ZAP

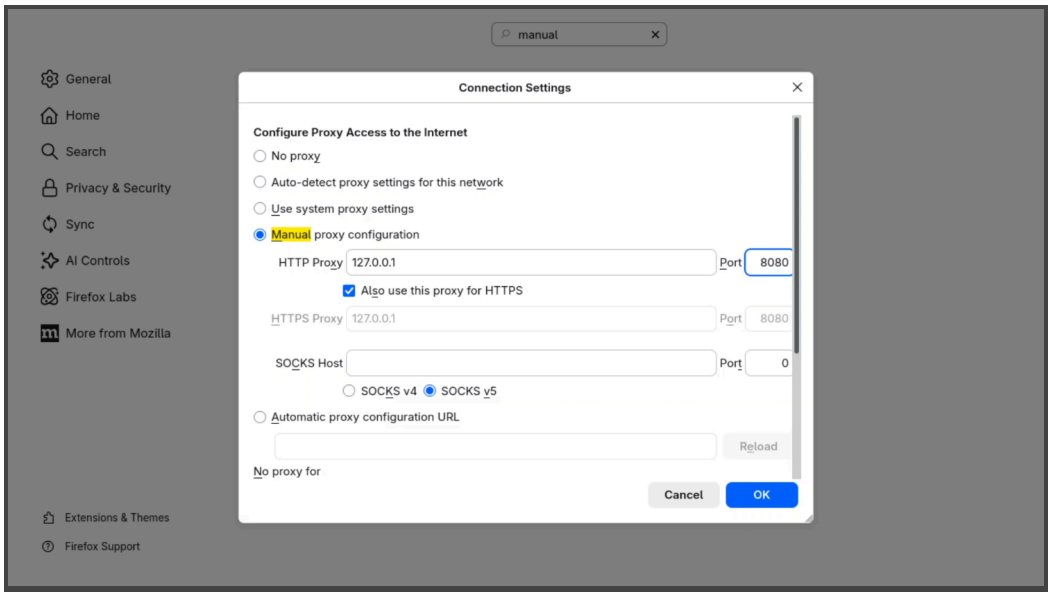
2.3.3 Linking OWASP ZAP to Firefox

After installing and running OWASP ZAP using flatpak, I met with the OWASP ZAP interface.



(Figure 2.3.1)

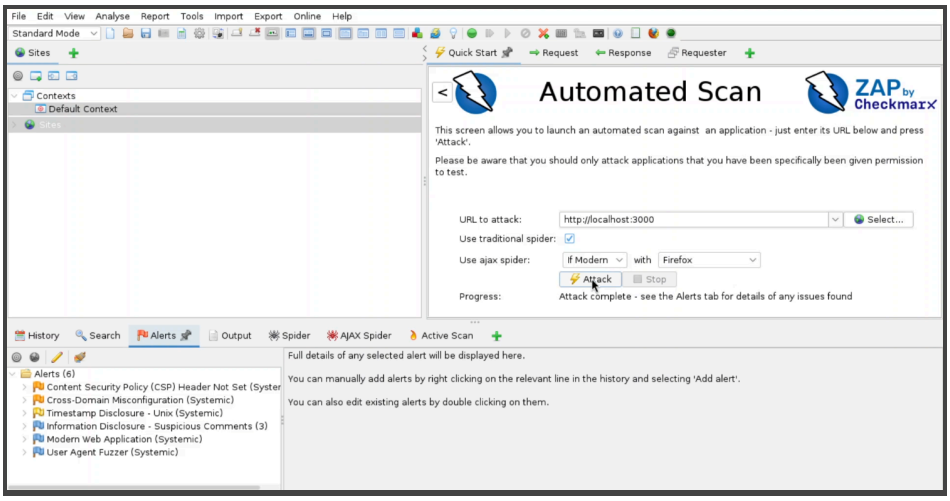
In order for OWASP ZAP to identify what browser I was wanting to use, I had to manually open firefox, go to Settings > General > Proxy Settings > Configure Proxy. This led me to the Connection Settings, where I changed “Use system proxy settings” to “Manual proxy configuration”. I then proceeded to add 127.0.0.1 (local host) as the HTTP Proxy, with port 8080, and checked the “Also use this proxy for HTTPS” box.



(Figure 2.3.2)

2.3.4 Initiating A Scan

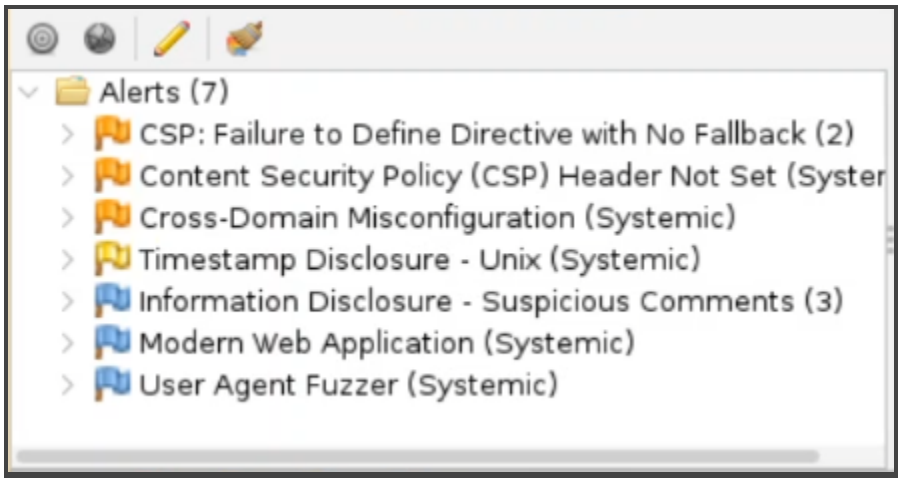
Since I was assigned to use an automated scan, I clicked on “Automated Scan” (Refer to Figure 2.3.1), then typed in *http://localhost:3000* for the URL to scan, checked the traditional spider box, and clicked “Attack”.



(Figure 2.3.3)

2.3.4 Creating A Report

After the scan was done, a menu appeared that showed all of the vulnerabilities of the localhost. In order to view all the contents of all the vulnerabilities scanned, I automatically created a report, which is a feature within OWASP ZAP. The following link attached contains the entire report: OWASP ZAP Report.pdf



(Figure 2.3.4)

3.0 Findings Documentation

During the security testing phase, several common vulnerabilities were identified, mainly related to input handling, SQL Injection risk, and weak authentication practices. These issues were discovered using manual testing techniques such as injecting XSS payloads and SQL Injection strings through form inputs and login fields. The main improvement can be adding stricter input validation using the validator library to ensure only properly formatted data is accepted. This can help reduce the risk of malicious or malformed input being processed by the application.

SQL Injection risks in the login query can be fixed by switching to parameterized queries, which prevents user input from being interpreted as executable SQL code. This would significantly improve database security. Password security could also be strengthened by implementing bcrypt hashing, ensuring that passwords are stored in a secure, non-reversible format instead of plain text. Some XSS behavior might still be observed, which is expected in this intentionally vulnerable training application. In real-world applications, this would be mitigated using proper output encoding and security headers.

Overall, the changes would improve input handling, database security, and password protection, making the application significantly more secure.